DEEP LEARNING · STUDY NOTES

Backpropagation & Optimizers

How a neural network learns, and how to make it learn faster

Simplified from handwritten class notes, with the missing parts filled in, diagrams added and every formula checked.

What is inside

PART 1
Backpropagation
Toy network, 6-step training loop, loss, chain rule, all 9 gradients, worked example

PART 2
Optimizers
Problems of gradient descent, Momentum, Adagrad, RMSProp / Adadelta, Adam

PART 3
Quick Revision
Formula sheet, viva questions, memory tricks, common mistakes

The whole story in 5 lines

1. **Predict**: push data forward through the network to get $\hat{y}$.
2. **Measure**: the loss tells us how wrong $\hat{y}$ is.
3. **Blame**: backpropagation finds how much each weight contributed to the error.
4. **Fix**: gradient descent nudges every weight to reduce the error.
5. **Fix it faster**: optimizers (Momentum, Adagrad, RMSProp, Adam) make step 4 smarter.

How to use these notes

Read each section top to bottom: **idea** → **formula** → **example** → **problem**.

Coloured boxes: teal = tip or analogy, orange = watch out, green = remember for exams, purple = extra topic added to complete the notes.

Part 1 · Backpropagation

1.1 The big idea in one minute

- A new network knows nothing: its weights and biases are just starting guesses.
- It makes a prediction, compares it with the true answer, and measures the error (the **loss**).
- **Backpropagation** works out how much each weight and bias is to blame for that error, by sending the error **backwards** through the network.
- **Gradient descent** then nudges each parameter slightly in the direction that reduces the error.

> **Analogy: studying for an exam**
> **Forward pass** = you write the exam. **Loss** = the marks you lose. **Backpropagation** = you check which topics cost you the most marks. **Update** = you revise those topics harder. Repeat for every mock test until your score stops improving.

1.2 The toy problem: predict a student's package

We have four students. From IQ and CGPA we want to predict the placement package (in LPA). Because the answer is a **number**, this is a **regression** problem.

IQ (x_1)	CGPA (x_2)	Package (y)
80	8	3
60	9	5
70	9	8
120	7	11

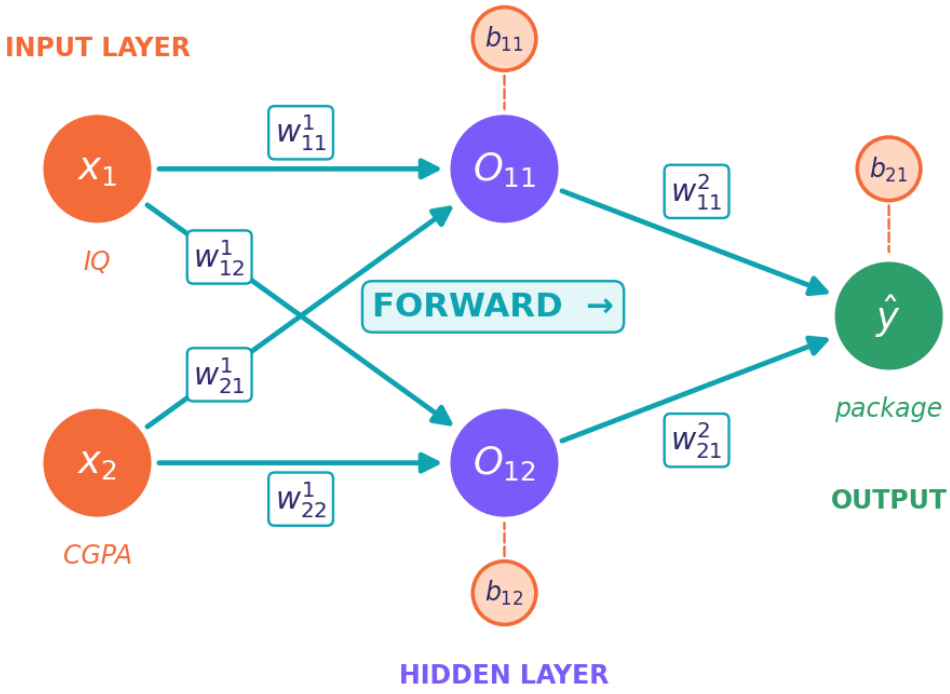


Fig 1. The network: 2 inputs → 2 hidden neurons → 1 output (orange circles = biases)

- **Input layer**: x_1 = IQ and x_2 = CGPA. **Hidden layer**: two neurons, O_{11} and O_{12}. **Output layer**: one neuron, $\hat{y}$.
- Learnable parameters: 4 weights w^1 + 2 hidden biases + 2 weights w^2 + 1 output bias = **9 parameters**.
- For simplicity we use a **linear activation** (a neuron just outputs weighted sum + bias).

> **Extra: why this simplification matters**
> With only linear activations, a stack of layers collapses into one linear function. Real networks use **ReLU, sigmoid or tanh**. The backprop idea is identical; one extra factor (the derivative of the activation) appears in each chain.

Notation (how to read the symbols)

Symbol	Meaning
x_1, x_2	Inputs: IQ and CGPA
w^1_{ij}	Layer-1 weight from input i to hidden neuron j (e.g. w^1_{21} = CGPA → O_{11})
w^2_{i1}	Layer-2 weight from hidden neuron i to the output
b_{11}, b_{12}, b_{21}	Biases of hidden neuron 1, hidden neuron 2 and the output neuron
O_{11}, O_{12}	Outputs of the hidden neurons
$\hat{y}$ and y	Predicted value and true value
L	Loss for one record
η (eta)	Learning rate (size of each step)

1.3 Key words

Term	Simple meaning
Weight	How strongly one input affects a neuron. Learned.
Bias	An adjustable offset that shifts a neuron's output. Learned.
Forward propagation	Pushing inputs through the network to get a prediction.
Loss function	Error for a **single** record.
Cost function	Error over **all** records (average of the losses).
Gradient	Slope of the loss with respect to a parameter: tells which way and how steeply the loss rises.
Learning rate η	How big a step we take against the gradient.
Epoch	One full pass through the entire training data.
Convergence	The loss has stopped improving: we are at (or near) a minimum.

1.4 The training loop: 6 steps

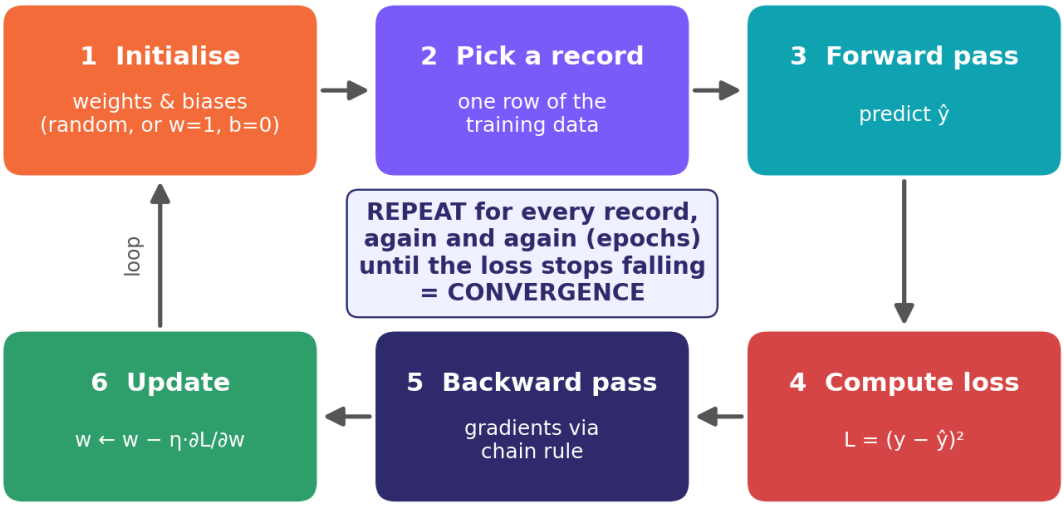


Fig 2. One training cycle. Steps 2 to 6 repeat for every record.

1. **Initialise** weights and biases: either random values, or simply all weights = 1 and all biases = 0.
2. **Select** a single record from the training dataset.
3. **Forward propagation**: predict the output ŷ.
4. **Calculate the loss** (how far ŷ is from y).
5. **Backward propagation**: compute the gradient of the loss with respect to every weight and bias.
6. **Update** weights and biases using gradient descent.

1.5 Step 3: forward propagation

Each neuron = (weights × inputs) + bias. For our network:

$$O_{11} = w^1_{11} \cdot x_1 + w^1_{21} \cdot x_2 + b_{11}$$

$$O_{12} = w^1_{12} \cdot x_1 + w^1_{22} \cdot x_2 + b_{12}$$

$$\hat{y} = w^2_{11} \cdot O_{11} + w^2_{21} \cdot O_{12} + b_{21}$$

1.6 Step 4: the loss function

- Our problem is **regression**, so we use **MSE** (Mean Squared Error). For one record: **L = (y − ŷ)²**.
- Example from the notes: for input (IQ 80, CGPA 8) the true package is y = 3, but the network predicted ŷ = 18.
- So L = (3 − 18)² = (−15)² = **225**. A big loss means a bad prediction.
- Squaring removes the minus sign and punishes big mistakes much more than small ones.
- Our job now: **reduce this loss** by changing the 9 parameters.

> **Remember: loss vs cost**
> **Loss** = error of one record. **Cost** = error of all records (usually their average). People often use the words loosely.

Extra: which loss for which problem?

Problem type	Typical loss
Regression (predict a number)	MSE (mean squared error)
Binary classification (yes / no)	Binary cross-entropy
Multi-class classification	Categorical cross-entropy

1.7 Step 5: backward propagation

The loss L depends on ŷ, and ŷ depends on everything before it. To reduce L we must know how each parameter affects it:

- L depends on ŷ.
- ŷ depends on w^2_{11}, w^2_{21}, b_{21}, O_{11} and O_{12}.
- O_{11} depends on w^1_{11}, w^1_{21}, b_{11}; O_{12} depends on w^1_{12}, w^1_{22}, b_{12}.

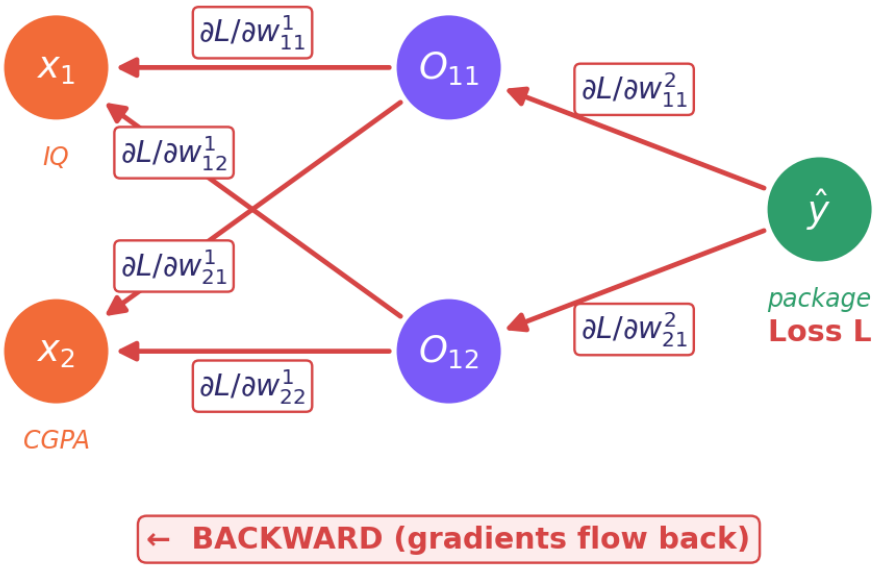


Fig 3. Backward pass: the error travels from the output back toward the inputs

> **Tip: the chain rule is a chain of gears**
> If gear A turns gear B and B turns gear C, then one turn of A turns C by (A→B ratio) × (B→C ratio). The chain rule multiplies slopes in exactly this way: **∂L/∂w = ∂L/∂ŷ × ∂ŷ/∂w**.

Derivation, step by step

Step A: the first link (same for every parameter).

$$L = (y - \hat{y})^2 \;\Rightarrow\; \partial L/\partial \hat{y} = -2(y - \hat{y})$$

Step B: output-layer parameters (one link more).

$$\partial L/\partial w^2_{11} = \partial L/\partial \hat{y} \cdot \partial \hat{y}/\partial w^2_{11} = -2(y - \hat{y})\cdot O_{11}$$
$$\partial L/\partial w^2_{21} = -2(y - \hat{y})\cdot O_{12}$$
$$\partial L/\partial b_{21} = -2(y - \hat{y})\cdot (1)$$

Why? $\hat{y} = w^2_{11}O_{11} + w^2_{21}O_{12} + b_{21}$; differentiating with respect to w^2_{11} leaves just O_{11}, and the other terms become 0.

Step C: hidden-layer parameters (two links more, going through O).

$$\partial L/\partial w^1_{11} = \partial L/\partial \hat{y} \cdot \partial \hat{y}/\partial O_{11} \cdot \partial O_{11}/\partial w^1_{11}$$
$$= -2(y - \hat{y}) \cdot w^2_{11} \cdot x_1$$

All 9 gradients at a glance

Parameter	Gradient (let $\delta = -2(y - \hat{y})$)	Layer
w^2_{11}	$\delta \cdot O_{11}$	Output
w^2_{21}	$\delta \cdot O_{12}$	Output
b_{21}	δ	Output
w^1_{11}	$\delta \cdot w^2_{11} \cdot x_1$	Hidden
w^1_{21}	$\delta \cdot w^2_{11} \cdot x_2$	Hidden
b_{11}	$\delta \cdot w^2_{11}$	Hidden
w^1_{12}	$\delta \cdot w^2_{21} \cdot x_1$	Hidden
w^1_{22}	$\delta \cdot w^2_{21} \cdot x_2$	Hidden
b_{12}	$\delta \cdot w^2_{21}$	Hidden

Watch out: a small correction to the handwritten notes

The notes wrote the first-layer gradients as $-2(y-\hat{y})\cdot w^1_{11}\cdot x$. Following the chain, the middle link is $\mathbf{w^2_{11}}$ (the weight on the path from O_{11} to $\hat{y}$), not w^1_{11}, and the input must be the correct one (x_1 for w^1_{11}, x_2 for w^1_{21}). The table above is the corrected version.

Also: in the update rule the left side is **b_new** (the notes wrote b_old).

1.8 Step 6: update with gradient descent

$$w_{new} = w_{old} - \eta \cdot \partial L/\partial w_{old}$$
$$b_{new} = b_{old} - \eta \cdot \partial L/\partial b_{old}$$
$$\eta = \text{learning rate}$$

- The **minus sign** moves us **downhill**: if the gradient is positive (loss grows as w grows) we decrease w, and vice versa.
- **η** controls the step size: too small = slow, too big = overshoot (see Part 2).
- All 9 parameters are updated together using their own gradients.

1.9 Worked example with small numbers

Setup: $x_1 = 1$, $x_2 = 2$, true $y = 3$, all weights = 1, all biases = 0, $\eta = 0.01$.

Stage	Calculation	Result
Forward	$O_{11} = 1 \cdot 1 + 1 \cdot 2 + 0$ $O_{12} = 1 \cdot 1 + 1 \cdot 2 + 0$ $\hat{y} = 1 \cdot 3 + 1 \cdot 3 + 0$	$O_{11} = 3$ $O_{12} = 3$ $\hat{y} = 6$
Loss	$L = (3 - 6)^2$	$L = 9$
Backward	$\delta = -2(3 - 6)$	$\delta = 6$
Gradients	w^2_{11}: $\delta \cdot O_{11}$ w^2_{21}: $\delta \cdot O_{12}$ w^1_{11}: $\delta \cdot 1 \cdot x_1$ w^1_{21}: $\delta \cdot 1 \cdot x_2$	18 18 6 12
Update	$w^2_{11} = 1 - 0.01 \cdot 18$ $w^1_{21} = 1 - 0.01 \cdot 12$ (all 9 updated the same way)	$w^2_{11} = 0.82$ $w^1_{21} = 0.88$
Forward again	New $\hat{y}$ with the updated parameters	$\hat{y} = 4.27$ $L = 1.61$

Result

In just **one update** the prediction moved from 6 to 4.27 (target 3) and the loss dropped from **9** to **1.61**. Repeating this many times drives the loss toward zero.

1.10 Repeat until convergence

- After the 9 parameters are updated by their gradients, the **next student record** is fed in.
- Predict with the modified parameters → compute the loss again → update again.
- Doing this over and over gives the **optimal weights and biases** where the loss is minimum. We say the model has **converged**.
- This whole process is called **backpropagation / loss minimisation**.

Extra: how many records per update?

Method	Records used per update	Character
Stochastic GD (SGD)	1 (what our notes did)	Fast, noisy updates
Mini-batch GD	A small batch, e.g. 32	Best balance; used in practice
Batch GD	All records	Smooth but slow and memory-hungry

Extra: two classic problems to know

Vanishing gradient: in deep networks the multiplied gradients become tiny, so early layers barely learn (common with sigmoid/tanh; ReLU helps).

Exploding gradient: the opposite, gradients grow huge and training blows up (fixed by gradient clipping and careful initialisation).

Part 2 · Optimizers

2.1 Why do we need optimizers?

- We already know gradient descent (GD): **w_new = w_old − η · gradient**.
- In real life plain GD has several problems, so we use **smarter versions of it, called optimizers**.

> **Analogy: walking down a foggy mountain**
> You can only feel the slope under your feet. Plain GD = take a fixed-size step downhill every time. Optimizers add **memory** (remember where you were heading), **adaptive steps** (small steps on steep, bumpy ground and big steps on flat ground) or both.

2.2 Problems with plain gradient descent

Problem 1: the learning-rate dilemma (η)

- If η is **small**: training takes a very long time and may get stuck in a **local minimum**.
- If η is **large**: the model can **overshoot** the global minimum and bounce around or even diverge.

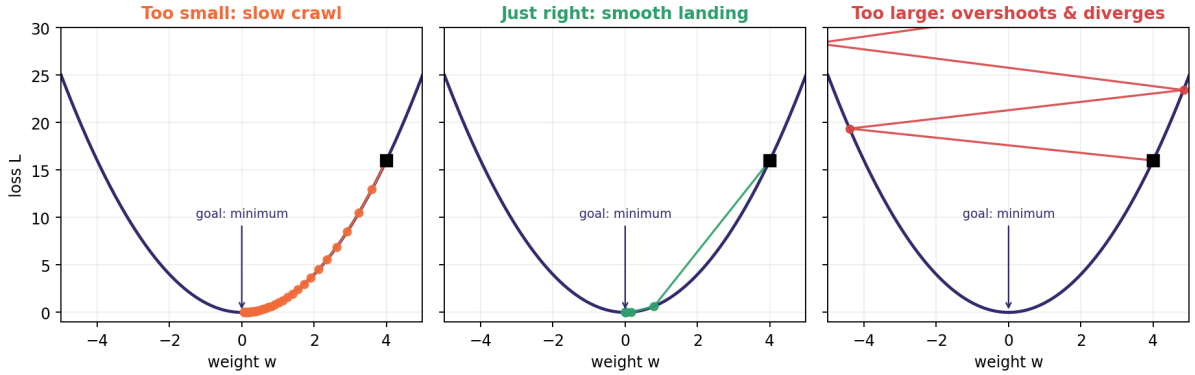


Fig 4. Same loss curve, three learning rates (loss = w²)

Problem 2: same learning rate for every parameter

Every weight is updated with the same η, but in reality **some parameters need bigger steps and some need smaller steps**.

Problem 3: saddle points and plateaus

At these points the gradient (partial derivative of the loss with respect to w and b) becomes **0**, which looks like convergence, but it is **not** the minimum. Training stalls.

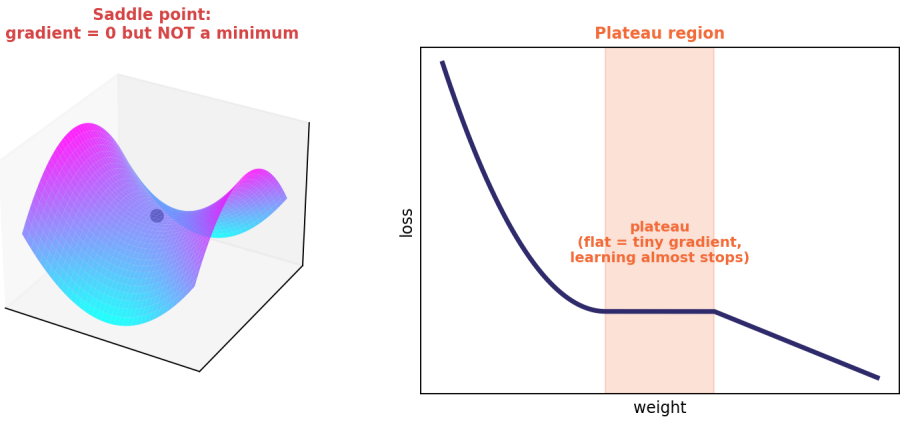


Fig 5. A saddle point is flat in one direction but a hill in another; a plateau has almost no slope

Problem	What goes wrong	Fixed by
Zig-zag, slow learning, flat regions	Each step only looks at the current gradient	Momentum
One learning rate for all weights	Some weights over-step, others crawl	Adagrad, RMSProp, Adadelta
Learning rate dying out	Adagrad's steps shrink too much	RMSProp, Adadelta
Everything at once	Needs speed and per-weight rates	Adam

2.3 Momentum gradient descent

In plain GD you only add the **current gradient** and decide the next step. This leads to: (1) **zig-zag** movement, (2) **slow learning**, (3) **getting stuck in flat regions**.

> **Analogy: a heavy ball rolling downhill**
> Push a heavy ball down a hill: it **builds speed** over time, and even when the slope becomes flat it **keeps moving**. That extra movement is **momentum**. So we update the parameters using the **current gradient and the previous gradients**.

We do not use the gradient directly. We introduce a new term, **velocity (v)** = past movement + current gradient:

$$v = \beta \cdot v_{old} + \eta \cdot \text{gradient}$$

$$w_{new} = w_{old} - v$$

β (beta, usually 0.9) = how much of the past velocity is kept.

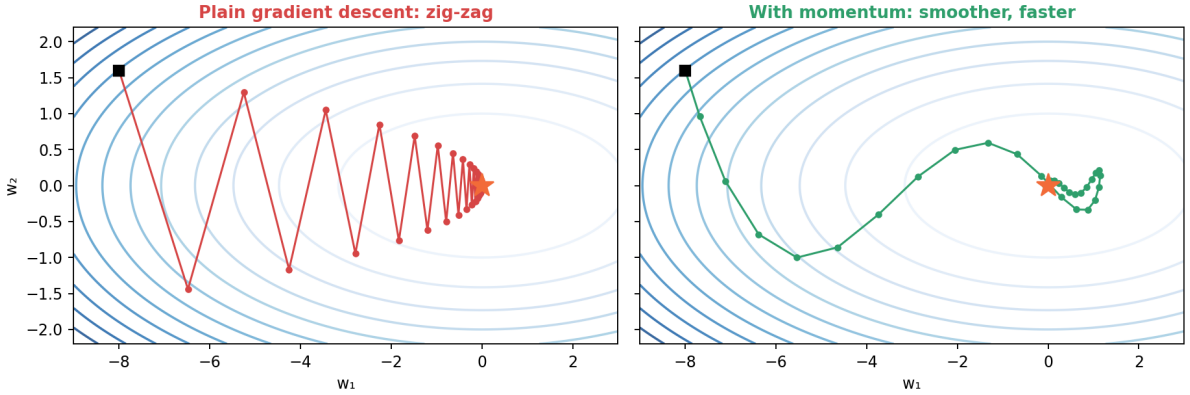


Fig 6. Momentum cancels the sideways zig-zag and speeds up along the valley

- **Zig-zag** steps point up, then down, so in the running total they **cancel out**.
- **Consistent** steps keep pointing the same way, so they **add up** and the ball speeds up.

Example from the notes

η = 0.1, β = 0.9, initial velocity v = 0, initial weight w = 10.

Step	Gradient	Without momentum	With momentum
1	5	update = 0.1 × 5 = 0.5 w = 10 − 0.5 = 9.5	v = 0.9×0 + 0.1×5 = 0.5 w = 10 − 0.5 = **9.5**
2	2	update = 0.1 × 2 = 0.2 w = 9.5 − 0.2 = 9.3	v = 0.9×0.5 + 0.1×2 = 0.65 w = 9.5 − 0.65 = **8.85**

Observation
Without momentum the step size was **0.2**; with momentum it was **0.65**. The previous push (0.5) is remembered, so the weight travels further even though the new gradient (2) is smaller.

2.4 Adagrad (Adaptive Gradient)

- **Idea**: give each parameter its **own learning rate**, which shrinks as that parameter accumulates large gradients.
- Frequently-updated weights get smaller steps; rarely-updated weights keep bigger steps (great for sparse data).

$$\text{update} = \eta \cdot \text{gradient} \,/\, \sqrt{(\text{ sum of past squared gradients })}$$

$$(\text{in code, a tiny } \varepsilon \approx 10^{-8} \text{ is added inside the root to avoid dividing by } 0)$$

Example from the notes

η = 0.1, initial weight = 10, and suppose the gradient is 2 at both steps.

Step	Calculation	New weight
1	update = 0.1 / √(2²) × 2 = 0.1 / 2 × 2 = **0.1**	10 − 0.1 = **9.9**
2	update = 0.1 / √(2² + 2²) × 2 = 0.1 / √8 × 2 ≈ **0.07**	9.9 − 0.07 = **9.83**

Observation and problem
The step size **decreases automatically** (0.1 → 0.07). That is useful at first, but the sum of squares only ever **grows**, so the learning rate keeps falling and eventually the updates become **very small**: learning slows to a crawl.

2.5 RMSProp and Adadelta

- **Solution**: take only a certain portion, i.e. use the **recent** gradients instead of all of them.
- Instead of a growing sum, keep a **decaying (exponential) average** of squared gradients, so old gradients are gradually forgotten.
- Adadelta is very similar to RMSProp, so both fix Adagrad's problem by focusing on recent gradients.

$$E_{new} = \beta \cdot E_{old} + (1 - \beta) \cdot \text{gradient}^2$$

$$\text{update} = \eta \cdot \text{gradient} \,/\, \sqrt{E_{new}}$$

Example from the notes

η = 0.1, decay factor β = 0.9 (so 1 − β = 0.1), initial average E = 0, gradient = 2 each step.

Step	Average of squared gradients	Update
1	E = 0.9×0 + 0.1×2² = 0.4	0.1 / √0.4 × 2 ≈ **0.32**
2	E = 0.9×0.4 + 0.1×2² = 0.36 + 0.4 = 0.76	0.1 / √0.76 × 2 ≈ **0.23**

Note: √0.4 ≈ 0.632 (the notes show 0.02 in the middle of the working, but the final answer 0.32 is right).

Observation
The step size decreases automatically, **but not as fast as Adagrad**. Adagrad would give 0.1 then 0.07; RMSProp gives 0.32 then 0.23 and settles near η instead of dying out.

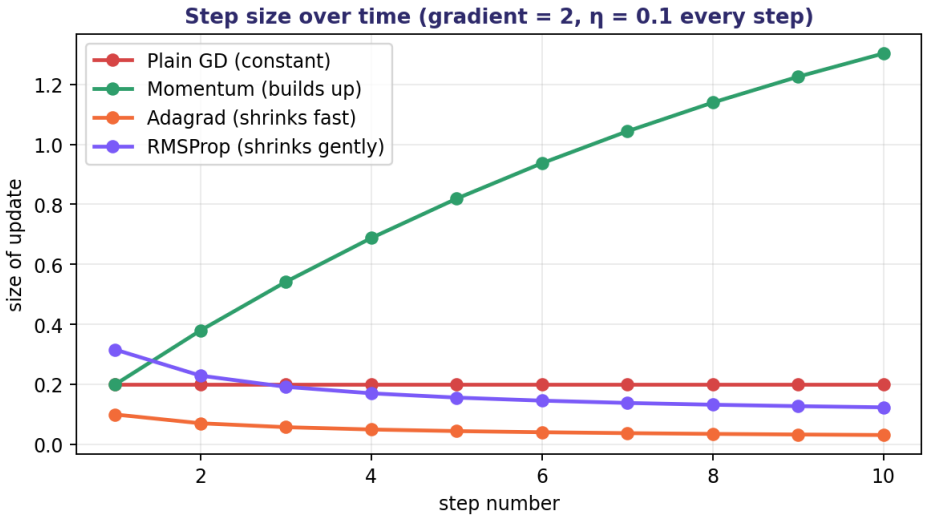


Fig 7. How the step size behaves for each method (constant gradient = 2)

2.6 Adam (Adaptive Moment Estimation)

- Adam **combines Momentum and RMSProp (Adagrad-style adaptive rates)** in one optimizer.
- It keeps two running averages: **m** (average of gradients = direction/momentum) and **v** (average of squared gradients = per-weight scaling).

$$m = \beta_1 \cdot m + (1 - \beta_1) \cdot g \qquad v = \beta_2 \cdot v + (1 - \beta_2) \cdot g^2$$

$$\hat{m} = m \,/\, (1 - \beta_1^t) \qquad \hat{v} = v \,/\, (1 - \beta_2^t)$$

$$w_{new} = w_{old} - \eta \cdot \hat{m} \,/\, (\sqrt{\hat{v}} + \varepsilon)$$

Hyper-parameter	Usual default	Role
η (learning rate)	0.001	Overall step size
β_1	0.9	Memory of gradients (momentum part)
β_2	0.999	Memory of squared gradients (RMSProp part)
ε	10^{-8}	Avoids division by zero

Mini example (first step) – extra

Gradient g = 2, t = 1, defaults above.

$$m = 0.1 \times 2 = 0.2 \qquad v = 0.001 \times 4 = 0.004$$

$$\hat{m} = 0.2 \,/\, 0.1 = 2 \qquad \hat{v} = 0.004 \,/\, 0.001 = 4$$

$$\text{update} = \eta \cdot 2 \,/\, \sqrt{4} = \eta = 0.001$$

Why the bias correction?

m and v start at 0, so in the first few steps they are pulled toward 0. Dividing by $(1 - \beta^t)$ fixes that. A nice side effect: the first step is about η whatever the gradient size.

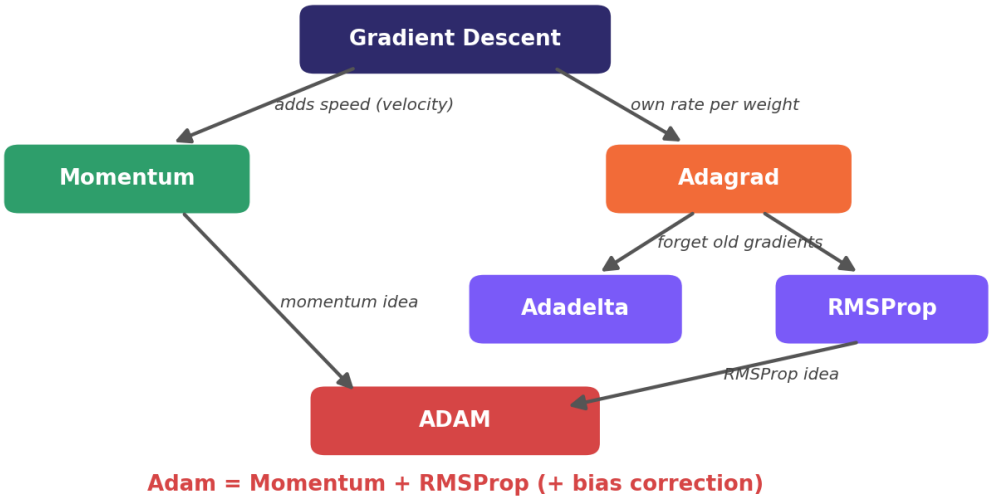


Fig 8. The optimizer family tree

2.7 Side-by-side comparison

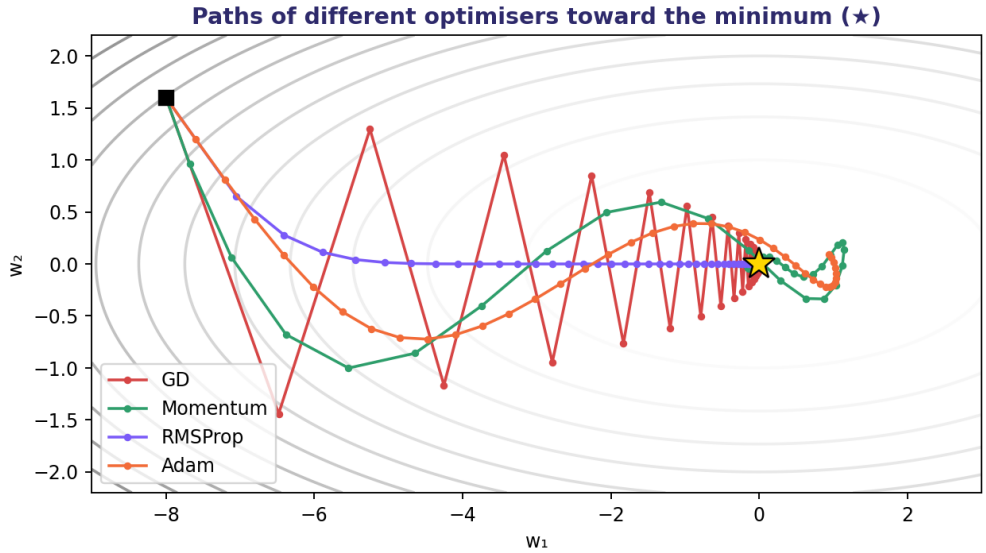


Fig 9. Same start, same bowl: how each optimizer travels (illustrative simulation)

Optimizer	Core idea	Fixes	Weakness
Gradient Descent	Step against the gradient	–	Zig-zag, slow, one η for all
Momentum	Add velocity (past + present)	Zig-zag, slow, flat regions	Can overshoot; one η for all
Adagrad	Divide by √(sum of all past g^2)	Per-weight rates	η shrinks until learning stops
RMSProp / Adadelta	Divide by √(decaying avg of g^2)	Adagrad's dying rate	Needs a tuned η and β
Adam	Momentum + RMSProp + bias correction	Most problems at once	Sometimes generalises slightly worse than SGD

2.8 Which one should I use?

- **Start with Adam** (η = 0.001): a strong default that works well with little tuning.

- **SGD + momentum**: often chosen for the best final accuracy in image models, but needs more tuning.
- **RMSProp**: a common choice for recurrent networks.
- **Adagrad**: handy for sparse features (e.g. text with rare words).

Using an optimizer in Keras:

```
import tensorflow as tf

# pick one optimizer
opt = tf.keras.optimizers.Adam(learning_rate=0.001)
# opt = tf.keras.optimizers.SGD(learning_rate=0.01, momentum=0.9)
# opt = tf.keras.optimizers.RMSprop(learning_rate=0.001)
# opt = tf.keras.optimizers.Adagrad(learning_rate=0.01)

model.compile(optimizer=opt, loss="mse")
model.fit(X, y, epochs=100, batch_size=32)
```

Part 3 · Quick Revision

3.1 Formula cheat sheet

Topic	Formula
Neuron output	$z = (w \cdot x) + b$ (linear activation: output = z)
Loss (MSE, one record)	$L = (y - \hat{y})^2$
First backward link	$\partial L/\partial \hat{y} = -2(y - \hat{y})$
Output weight gradient	$\partial L/\partial w^2_{11} = -2(y - \hat{y}) \cdot O_{11}$
Hidden weight gradient	$\partial L/\partial w^1_{11} = -2(y - \hat{y}) \cdot w^2_{11} \cdot x_1$
Gradient descent	$w = w - \eta \cdot \partial L/\partial w$
Momentum	$v = \beta v + \eta g$; $w = w - v$
Adagrad	$w = w - \eta g / \sqrt{(\Sigma g^2)}$
RMSProp	$E = \beta E + (1-\beta) g^2$; $w = w - \eta g / \sqrt{E}$
Adam	$w = w - \eta \cdot \hat{m} / (\sqrt{\hat{v}} + \varepsilon)$

3.2 Likely viva / exam questions

Question	Short answer
What is backpropagation?	Algorithm that computes the gradient of the loss for every weight by applying the chain rule backwards, then updates them with gradient descent.
Loss vs cost function?	Loss is the error for one record; cost is the error over all records.
Why square the error in MSE?	Removes the sign and penalises large errors more.
Why a minus in the update rule?	To move opposite to the gradient, i.e. downhill.
Name 3 problems of plain GD.	Learning-rate choice, same rate for all parameters, saddle points / plateaus (also zig-zag and slow learning).
How does momentum help?	Velocity remembers past gradients: it cancels zig-zag, speeds up, and carries across flat regions.
Adagrad's main drawback?	Squared gradients accumulate forever, so the learning rate decays to almost zero.
How does RMSProp fix it?	Uses a decaying average of recent squared gradients instead of the full sum.
What is Adam?	Adaptive Moment Estimation: momentum + RMSProp with bias correction.

3.3 Memory tricks

Momentum	Adagrad	RMSProp	Adam
remembers the direction	**accumulates** everything	**forgets** old gradients	does **both**: remembers and forgets

3.4 Common mistakes checklist

- Forgetting that **hidden-layer gradients pass through the next layer's weight** (w^2).

- Updating the parameters one at a time and recomputing the loss in between. Compute **all gradients first**, then update.
- Using a learning rate that is too large and seeing the loss go **up** instead of down.
- Thinking gradient = 0 always means a minimum (it can be a saddle point or plateau).
- Mixing up **loss** (one record) and **cost** (all records).

> **You made it!**
> If you can redraw Fig 2 (the 6-step loop) from memory and write the Adam update, you have the core of this chapter.